

Below are **two demo practice tasks designed with the same structure, difficulty level, and evaluation logic**. These are aligned to help students build **cross-application, real-time system design skills** before the Agumentik physical campus drive.

◆ **Demo Practice Task 1: Real-Time Order Processing & Dispatch System**

Total Duration: 4 Hours

Modules: 4 (1 Hour Each)

■ **Important Rule:** If Module 1 is not completed within the first 2 hours, you cannot proceed further.

■ **Application Type:** You must build both applications:

Customer Application – Mobile App

Platform: React Native / Flutter

Purpose: Customers place orders and track delivery status

Admin Application – Web App

Platform: React / Flutter (Responsive)

Purpose:

- Manage incoming orders
 - Assign delivery partners
 - Track order lifecycle
-

◆ **Module 1: Order Queue System**

Backend

Create a system that:

- Stores orders in a queue
- Each order must include:
 - Order ID
 - Timestamp
 - Order type (standard / express)
 - Priority score
 - Status (pending, assigned, delivered)

Queue Logic

- Express orders have higher base priority
- Priority increases over time
- If equal priority → earlier order first

Customer Mobile App

Users must be able to:

- Place an order
- View:
 - Order ID

- Status
- Queue position

Admin Web App

Display:

- Order queue
- Priority
- Status

■ Mandatory Behavior

- Queue must auto-reorder
- Priority must dynamically increase
- No manual refresh

◆ Module 2: Delivery Partner Assignment

Backend

Add delivery partners with zones:

- North / South / East / West

Rules

- Orders assigned based on location
- If no partner available:
 - Order must wait
 - No queue skipping allowed

Admin Web App

Admin can:

- View delivery partners
- Mark availability
- Assign orders

■ Mandatory Behavior

- Correct partner assignment
- Queue order must not break

◆ Module 3: Real-Time Updates

Backend

Implement real-time updates using:

- WebSockets / Socket.io

Customer App

Auto-update when:

- Order is placed
- Assigned

- Delivered

Admin App

Display:

- Live orders
- Delivery activity

■ Mandatory Behavior

- Updates within 1 second
- No refresh allowed

◆ Module 4: Order Timeout & Cleanup

Backend

- If order is not accepted within X time:
 - Mark as expired
 - Remove from queue

Apps Behavior

- Real-time removal
- Queue recalculation

■ Mandatory Behavior

- No stale orders
- Queue consistency maintained

◆ Demo Practice Task 2: Smart Appointment Booking & Queue System

Total Duration: 4 Hours

Modules: 4 (1 Hour Each)

■ Important Rule: Module 1 must be completed within 2 hours to continue.

■ Application Type

User Application – Mobile App

Platform: React Native / Flutter

Purpose: Book appointments & track position

Admin Application – Web App

Platform: React / Flutter

Purpose:

- Manage appointment queue
- Assign service providers

◆ Module 1: Appointment Queue System

Backend

Create a system that:

- Stores appointments in queue

Each appointment must include:

- Appointment ID
- Timestamp
- Service type (consultation / support)
- Priority score
- Status

Queue Logic

- Emergency appointments get higher priority
- Priority increases over time
- FIFO if priority same

User Mobile App

Users must:

- Book appointment
- View:
 - Queue position
 - Estimated wait time

Admin Web App

Display:

- Appointment queue
- Priority
- Status

■ Mandatory Behavior

- Auto-priority increase
- Queue must always be accurate

◆ Module 2: Service Provider Mapping

Backend

Add service providers:

- Consultation specialist
- Support specialist

Rules

- Appointment must match provider type
- If unavailable:
 - Appointment waits
 - No bypass allowed

Admin Web App

Admin can:

- View providers
- Mark availability
- Assign appointments

■ **Mandatory Behavior**

- Correct mapping only
- Queue integrity maintained

◆ **Module 3: Live Queue Synchronization**

Backend

Enable real-time updates

User App

Auto-update when:

- New appointment added
- Assigned
- Completed

Admin App

Show:

- Live queue
- Provider activity

■ **Mandatory Behavior**

- <1 sec update latency
- No refresh

◆ **Module 4: Activity Monitoring (Heartbeat System)**

Backend

- Each active user sends heartbeat
- If no activity:
 - Remove appointment
 - Recalculate queue

Apps Behavior

- Auto heartbeat from mobile
- Real-time removal

■ **Mandatory Behavior**

- No inactive users in queue
- Queue remains consistent

■ **Quick Cheat Sheet for Students**

■ **Core Concepts to Focus:**

- Priority Queue Logic (Dynamic ranking)

- Real-time sync (Socket.io / WebSockets)
 - Backend-driven state (NO frontend hacks)
 - System constraints (no bypass, no refresh)
-

◆ **Suggested Tech Setup:**

- Backend: Node.js + Express
 - DB: MongoDB / In-memory
 - Realtime: Socket.io
 - Mobile: React Native
 - Web: React (Responsive)
-

■ **Common Mistakes to Avoid:**

- Hardcoding queue updates
- Breaking queue order
- Ignoring edge cases (no agent available)
- Using refresh instead of realtime
- Skipping modules